

FAST – NUCES



Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

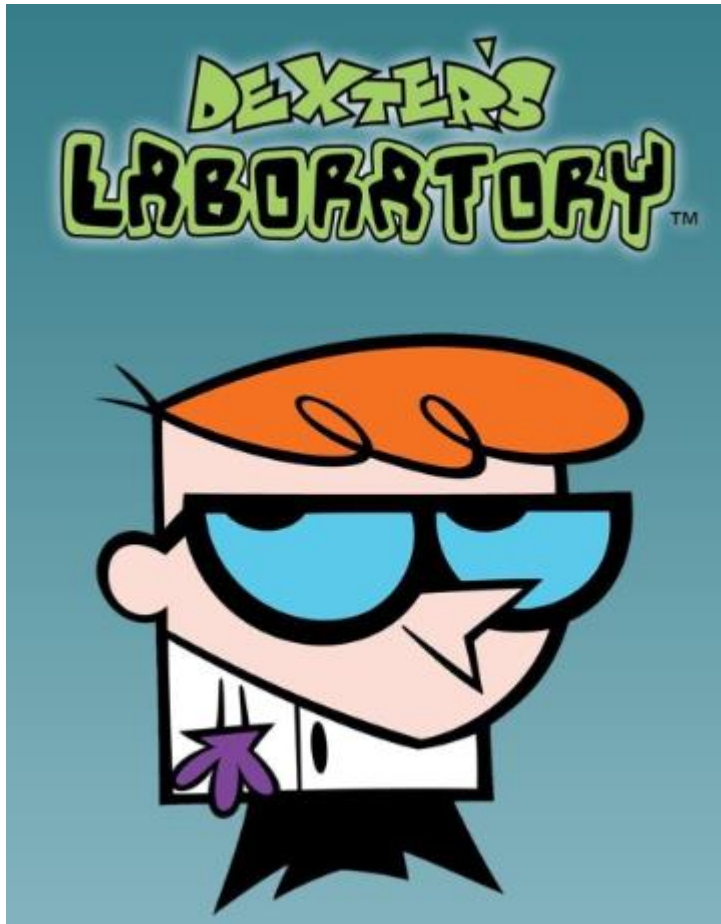
Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 1 – Introduction to Dynamic Programming

Objectives

- Getting started with dynamic programming
- Understanding and implementing pointers



Read carefully and attempt following tasks

Dexter's Laboratory – Pointer Experiments

Before starting the experiments below, carefully observe and understand the following code and its output. You are NOT allowed to move to the next experiment without understanding how pointers work here.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int* p = &a;

    cout << "a = " << a << endl;
    cout << "&a = " << &a << endl;
    cout << "p = " << p << endl;
    cout << "*p = " << *p << endl;

    *p = 25; // modify 'a' through pointer
    cout << "a after *p = 25 → " << a << endl;

    return 0;
}
```

Dexter's Note:

If you don't understand how a, p, &a, and *p are related, **STOP HERE** and revise pointers before proceeding.



1. Dexter's Addition Machine

Dexter is building a machine that takes **two numbers** and stores their **sum**.

- Take two integers as input.
- Store their sum in a third variable.
- Display the **memory addresses** where:
 - the first number
 - the second number
 - the sumare stored.

2. Dee Dee's Number Switcher (Pointers Only!)

Dee Dee accidentally swapped two control values in Dexter's lab!

Declare two integers a and b and initialize them from user input.

Use **pointers only** to swap their values.

- Do **not** use a third variable directly.

After swapping:

- Multiply the swapped values using pointers.
- Store the result in a new variable.
- Display:
 - swapped values
 - multiplication result

3. Dexter's Remote Swap Function

Dexter wants a reusable swap function for his experiments.

- Write a function that swaps two integers.
- Use **pass by reference**.
- Call the function from main() and verify the result.

4. Power Reset Button (Pointer Return)

Dexter installs an emergency **RESET button** on a machine.

- Write a function that:
 - takes a pointer to an integer
 - resets its value to 0
 - returns the **same pointer**
- Prototype:
int* resetToZero(int* p);
- In main():
 - declare an integer
 - pass its address to the function
 - display the updated value using the returned pointer

5. Cube Generator (Two Techniques)

Dexter tests two different cube generators.

- Write **two separate functions**:
 void cubeByPtr(int* p);
 void cubeByRef(int& r);
- One function must use a **pointer**
- The other must use a **reference**
- Call both from main() on different variables

6. Reference Booster

Dexter designs a booster that increases power levels.

Write a function that:

- takes an integer **by reference**
- increments it by 10
- returns the same integer **by reference**

In main():

- display the original value
- call the function
- display the value after the function call
- modify the returned reference again and observe the effect

Understanding use of const in pointer

Concept Study: Const in Dexter's Lab

Dexter labels some wires as **DO NOT TOUCH**.

Pointer Type	Declaration	Can change pointer?	Can change value?
Pointer to const	const int* pointer1	✓ Yes	✗ No
Const pointer	int* const pointer2	✗ No	✓ Yes
Const pointer to const	const int* const pointer3	✗ No	✗ No

7. Protected Memory Test

Dexter sets security rules on his variables.

- Read three integers from the user: a, b, and c.
- Create three pointers:
 - ptr1: pointer to **const int** pointing to a
 - ptr2: **const pointer** to int pointing to b
 - ptr3: **const pointer to const int** pointing to c
- Using these pointers:
 - Print all values
 - Attempt **invalid operations** such as:
 - modifying a const value
 - changing a const pointer
- Write these invalid lines **but comment them out**
- Clearly **observe and explain**:
 - which operations are allowed
 - which are not and why

8. Lab Disaster – Memory Leaks & Dangling Pointers

Dee Dee messed up Dexter's memory management!

Given the following code:

```
#include <iostream>
using namespace std;

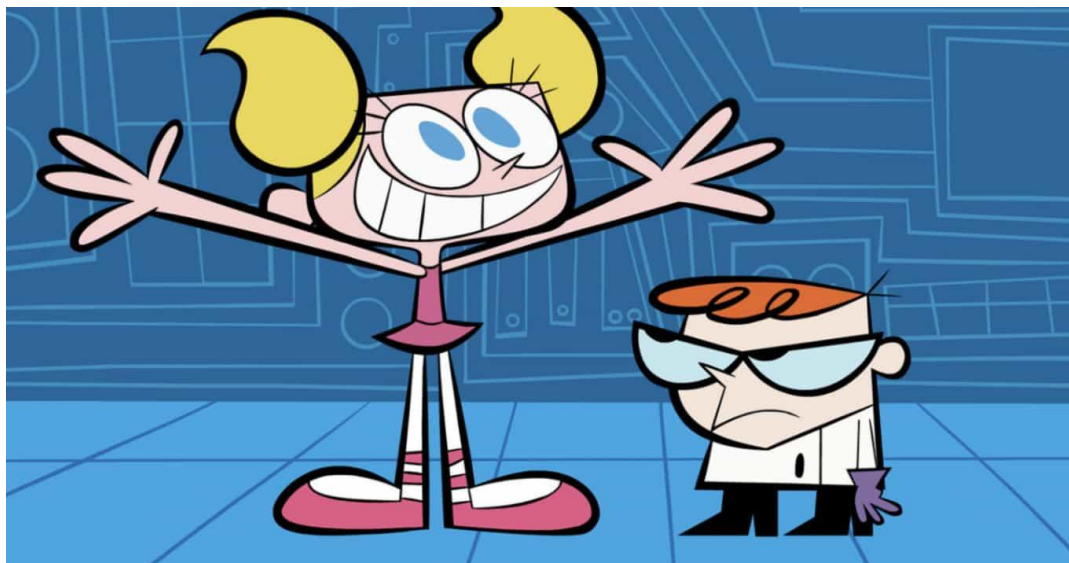
int main() {
    int* a = new int(100);
    int* b = a;
    delete a;

    int* c = new int(200);
    c = new int(300);

    return 0;
}
```

- Identify:
 - memory leaks
 - dangling pointers
- Rewrite the code to:
 - fix all memory issues
 - follow proper dynamic memory management rules

-----Happy Coding 😊-----





Submission Instructions

- Submit your tasks on Google Classroom within given deadline
- Submit solution file of each task separately for example “LAB01-TASK01.cpp”, “LAB01-TASK02.cpp” and so on
- Also submit screenshots of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
- “.cpp” is the actual file where all your code is saved.
- Always submit .cpp files when asked for submission
- Example path of .cpp file is

Desktop\Your_Roll_No\LAB01-TASK01\LAB01-TASK01\LAB01-TASK01.cpp

Copy your files from this path
